Type&Price

In [112…

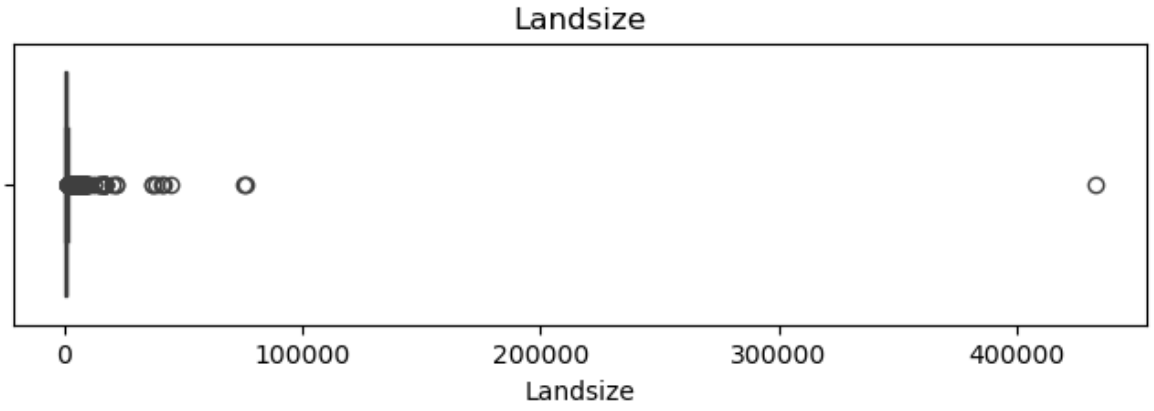
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("melb_data (6).csv")
print(df.shape)

type_map = {
    "h": "house",
    "u": "unit",
    "t": "townhouse"
}

df["Type"] = df["Type"].replace(type_map)

print(df["Type"].unique())
```

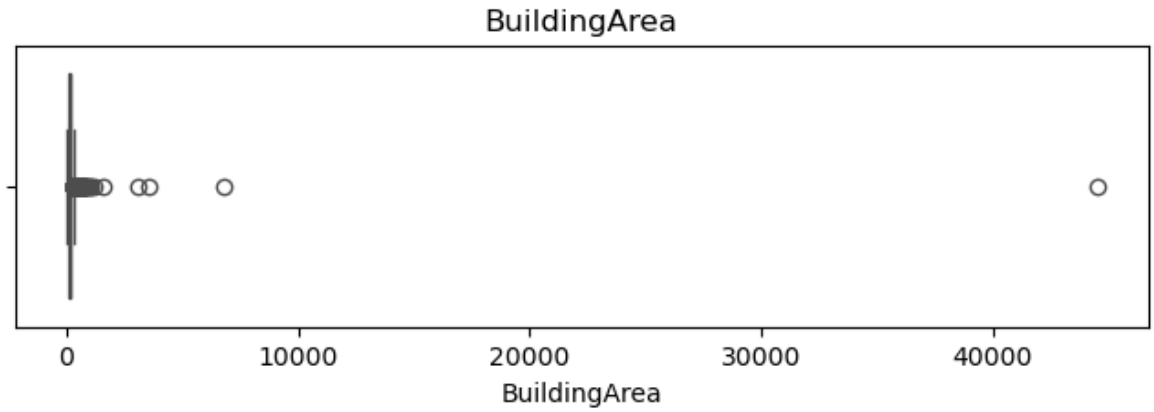
```
(13580, 21)
['house' 'unit' 'townhouse']
```

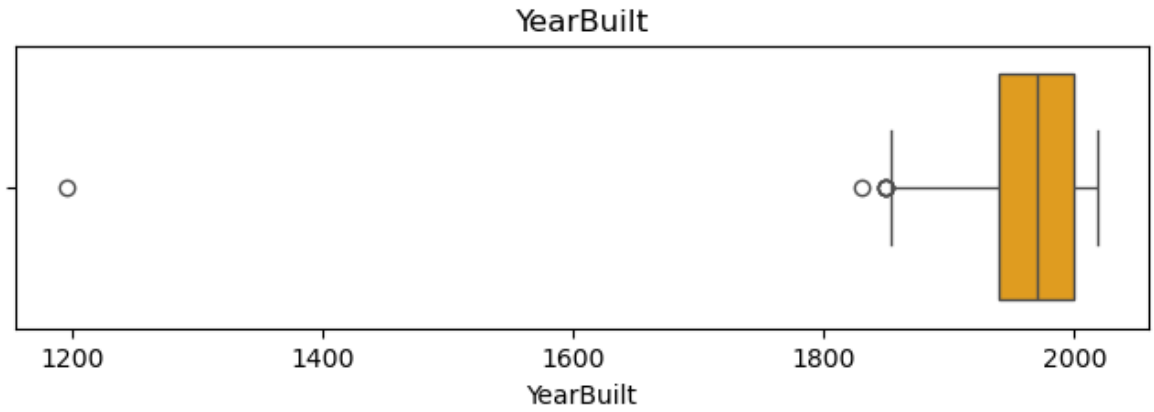
In [113…

```
# Visual Check 1: Landsize
plt.figure(figsize=(8, 2))
sns.boxplot(data = df, x='Landsize')
plt.title('Landsize')
plt.show()

# Visual Check 2: BuildingArea
plt.figure(figsize=(8, 2))
sns.boxplot(data = df, x='BuildingArea', color='orange')
plt.title('BuildingArea')
plt.show()

# Visual Check 3: YearBuilt
plt.figure(figsize=(8, 2))
sns.boxplot(data = df, x='YearBuilt', color='orange')
plt.title('YearBuilt')
plt.show()

# missing values per column
missing_counts = df.isnull().sum()

# precentage of missing values
missing_percentages = (df.isnull().sum() / len(df)) * 100

# Table of missing count and percentage
missing_summary = pd.DataFrame(
    {'Missing Count': missing_counts,
    'Percentage (%)': missing_percentages})

# shows the varibles that has missing values in decending order
final_report = missing_summary[missing_summary['Missing Count'] > 0].sort_values

print(final_report)
```

```python
# Assume missing car spaces mean 0 parking spots available
df['Car'] = df['Car'].fillna(0)

# checks if there is still missing values
car = df['Car'].count()
print(car)

# Drops rows where BuildingArea and YearBuilt are missing
df_clean = df.dropna(subset=['BuildingArea', 'YearBuilt'])

# Remove impossible 0 square metre entries
df_clean = df_clean[df_clean['Landsize'] > 0]
df_clean = df_clean[df_clean['BuildingArea'] > 0]

# Fix the Date data type mismatch so it plots chronologically
df_clean['Date'] = pd.to_datetime(df_clean['Date'], format='%d/%m/%Y', errors='c

print("Check")
print("Number of properties with 0 Landsize left:", (df_clean['Landsize'] == 0).
print("Number of properties with 0 Building area left:", (df_clean['BuildingArea
print("Number of missing values left in Car column:", df_clean['Car'].isnull().s
print("Total rows remaining in cleaned dataset:", len(df_clean))

# Visual Check 1: Landsize
plt.figure(figsize=(8, 2))
sns.boxplot(data=df_clean, x='Landsize', color='orange')
plt.title('Visual Check: Outliers in Landsize')
plt.show()

# Visual Check 2: BuildingArea
plt.figure(figsize=(8, 2))
sns.boxplot(data=df_clean, x='BuildingArea', color='orange')
plt.title('Visual Check: Outliers in BuildingArea')
plt.show()

# Visual Check 3: YearBuilt
plt.figure(figsize=(8, 2))
sns.boxplot(data=df_clean, x='YearBuilt', color='orange')
plt.title('Visual Check: Outliers in YearBuilt')
plt.show()
```

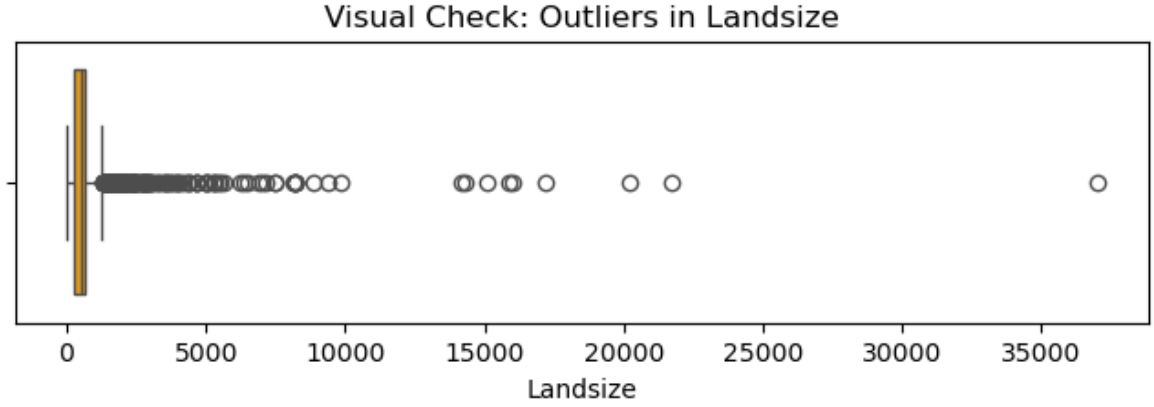


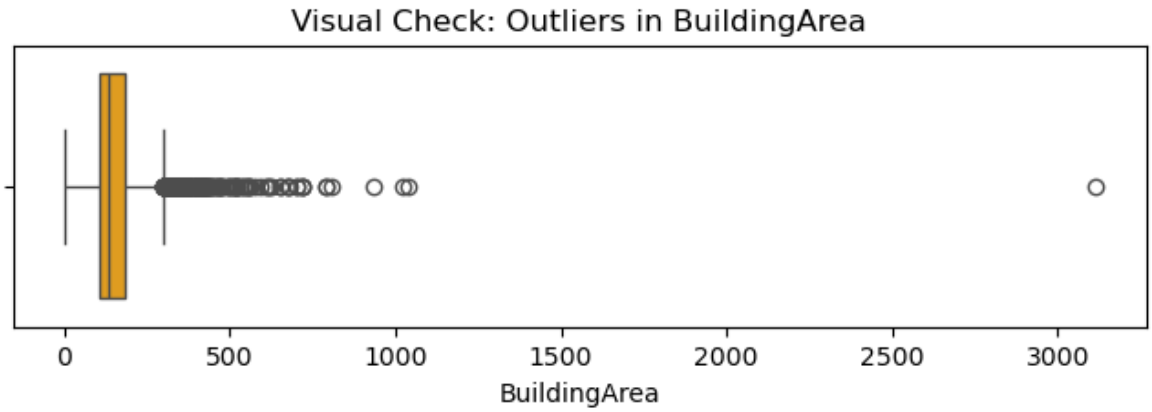


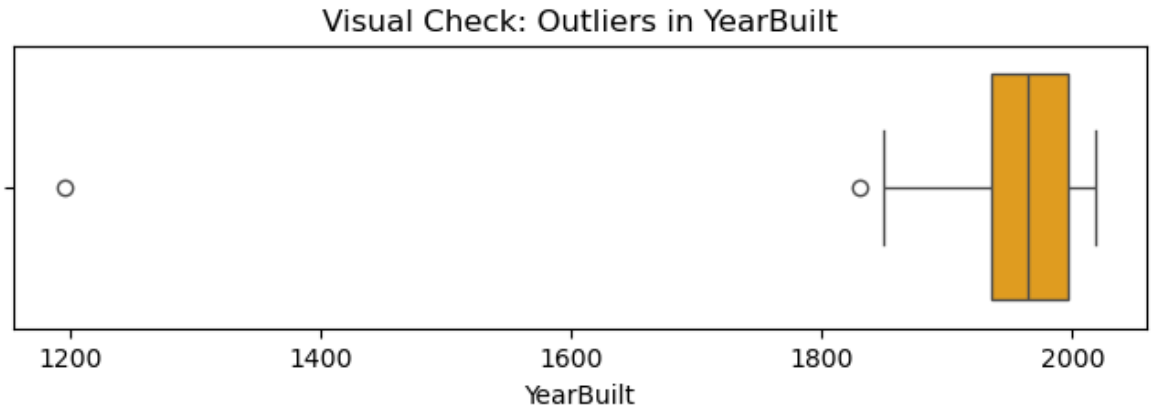


	Missing Count	Percentage (%)
BuildingArea	6450	47.496318
YearBuilt	5375	39.580265
CouncilArea	1369	10.081001
Car	62	0.456554

13580
Check
Number of properties with 0 Landsize left: 0
Number of properties with 0 Building area left: 0
Number of missing values left in Car column: 0
Total rows remaining in cleaned dataset: 5826

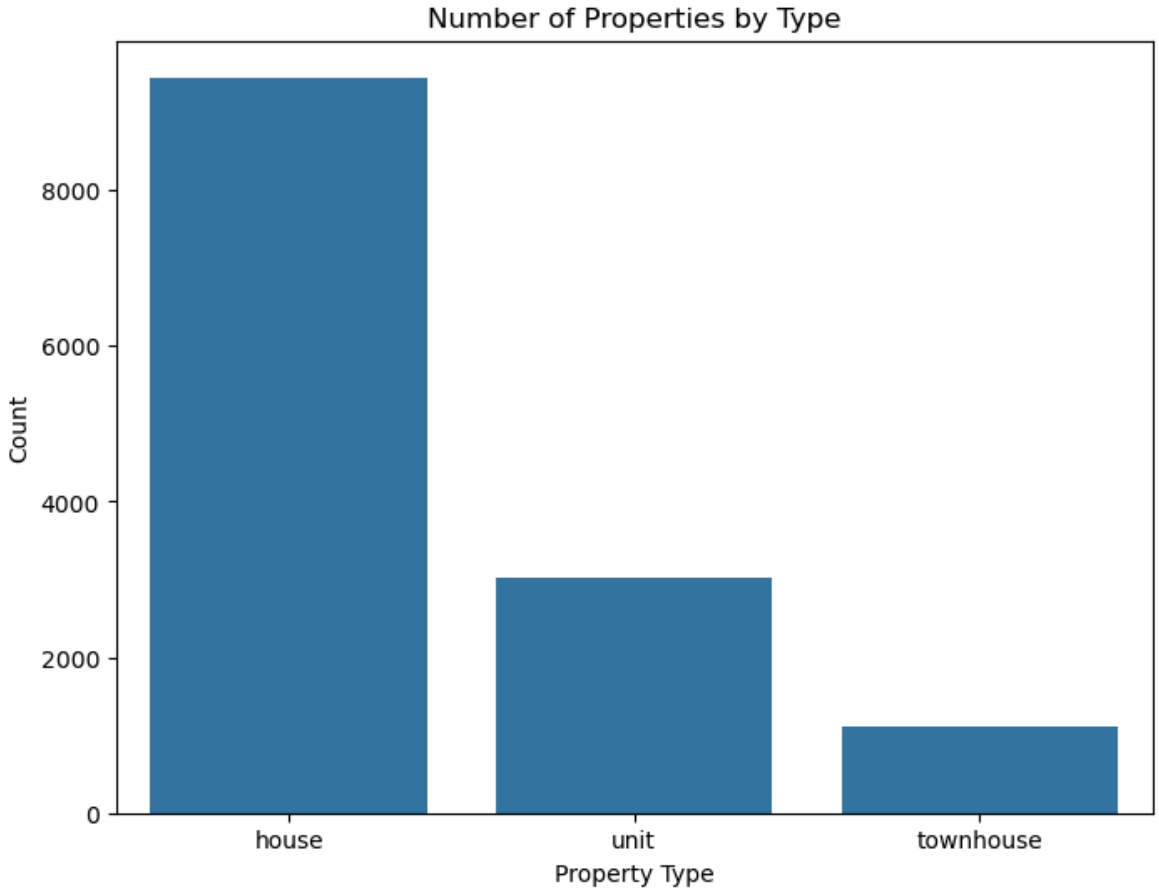






In [114…

```
plt.figure(figsize=(8, 6))
sns.countplot(data=df, x="Type", order=["house", "unit", "townhouse"])
plt.title("Number of Properties by Type")
plt.xlabel("Property Type")
plt.ylabel("Count")
plt.show()
```



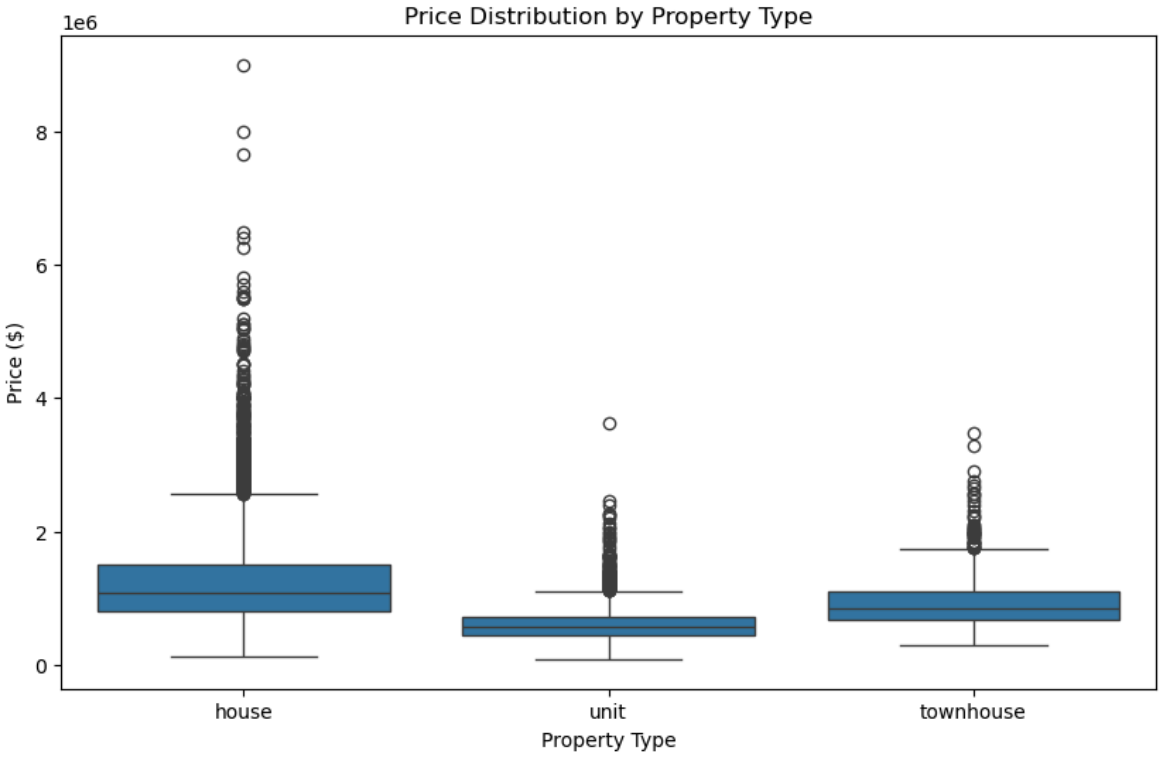
In [115…

```
type_counts = df["Type"].value_counts()
type_percent = df["Type"].value_counts(normalize=True) * 100

type_distribution = pd.DataFrame({
    "Count": type_counts,
    "Percentage": type_percent.round(2)
})

print(type_distribution)
```

```
           Count  Percentage
Type
house       9449       69.58
unit        3017       22.22
townhouse   1114        8.20
```

In [116…

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x="Type", y="Price", order=["house", "unit", "townhouse"])
plt.title("Price Distribution by Property Type")
plt.xlabel("Property Type")
plt.ylabel("Price ($)")
plt.show()
```



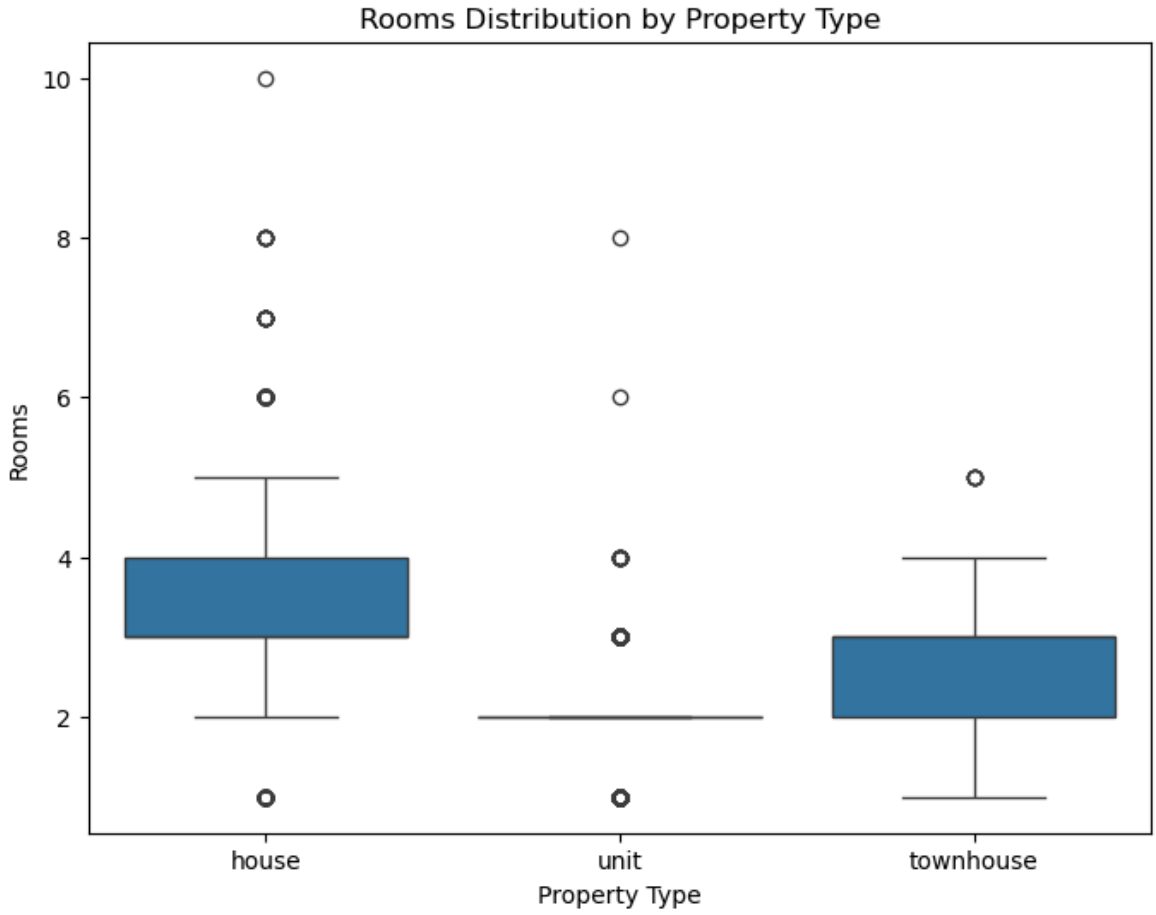
In [117…

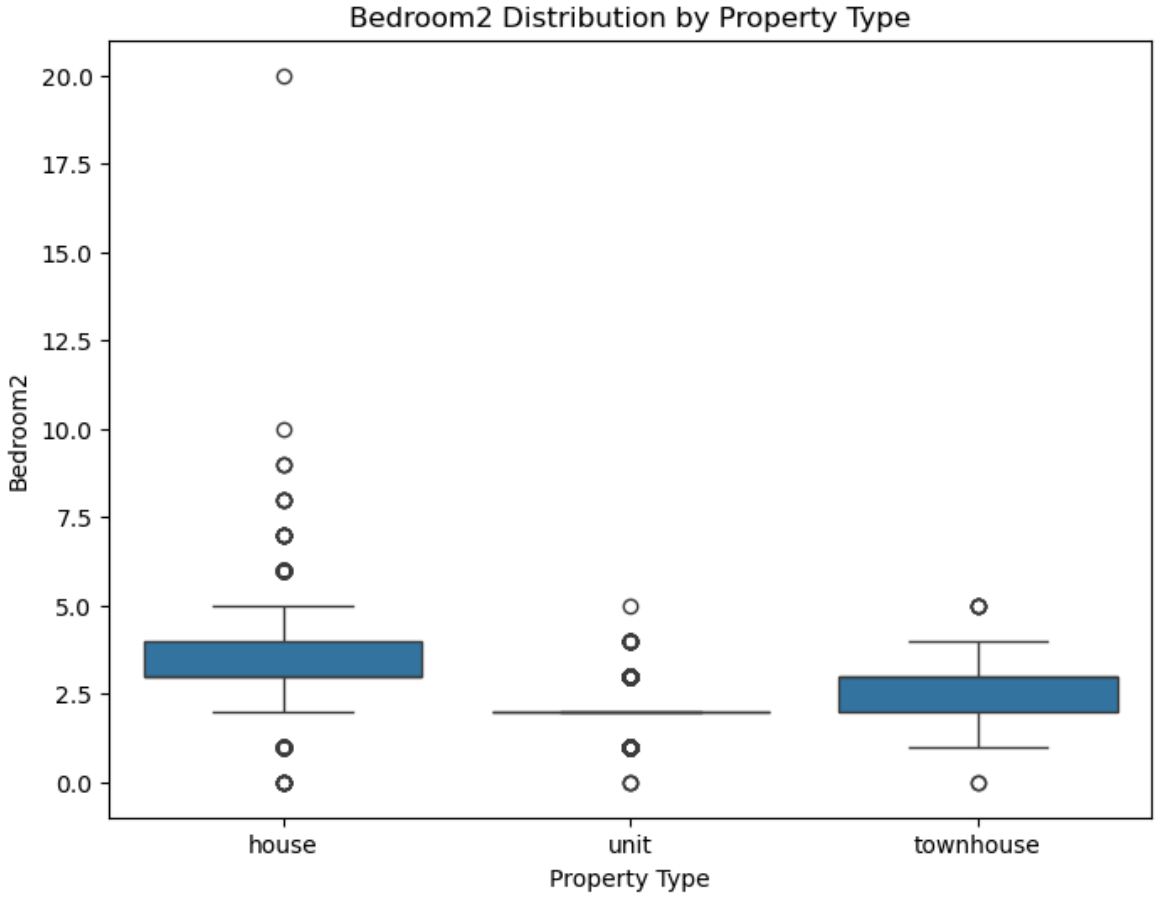
```
price_by_type = df.groupby("Type")["Price"].agg(["count", "mean", "median", "min
print(price_by_type)
```

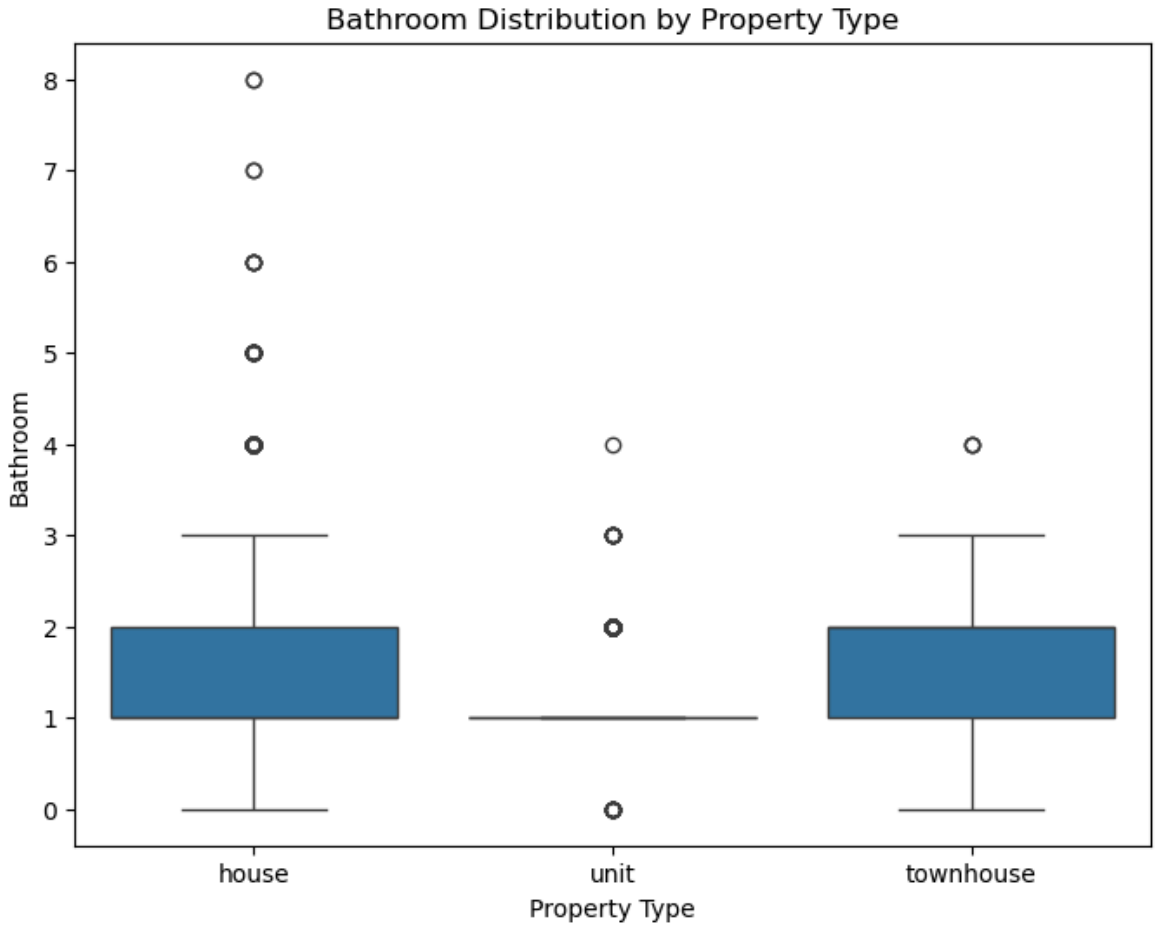
Type	count	mean	median	min	max
house	9449	1.242665e+06	1080000.0	131000.0	9000000.0
townhouse	1114	9.337351e+05	846750.0	300000.0	3475000.0
unit	3017	6.051275e+05	560000.0	85000.0	3625000.0

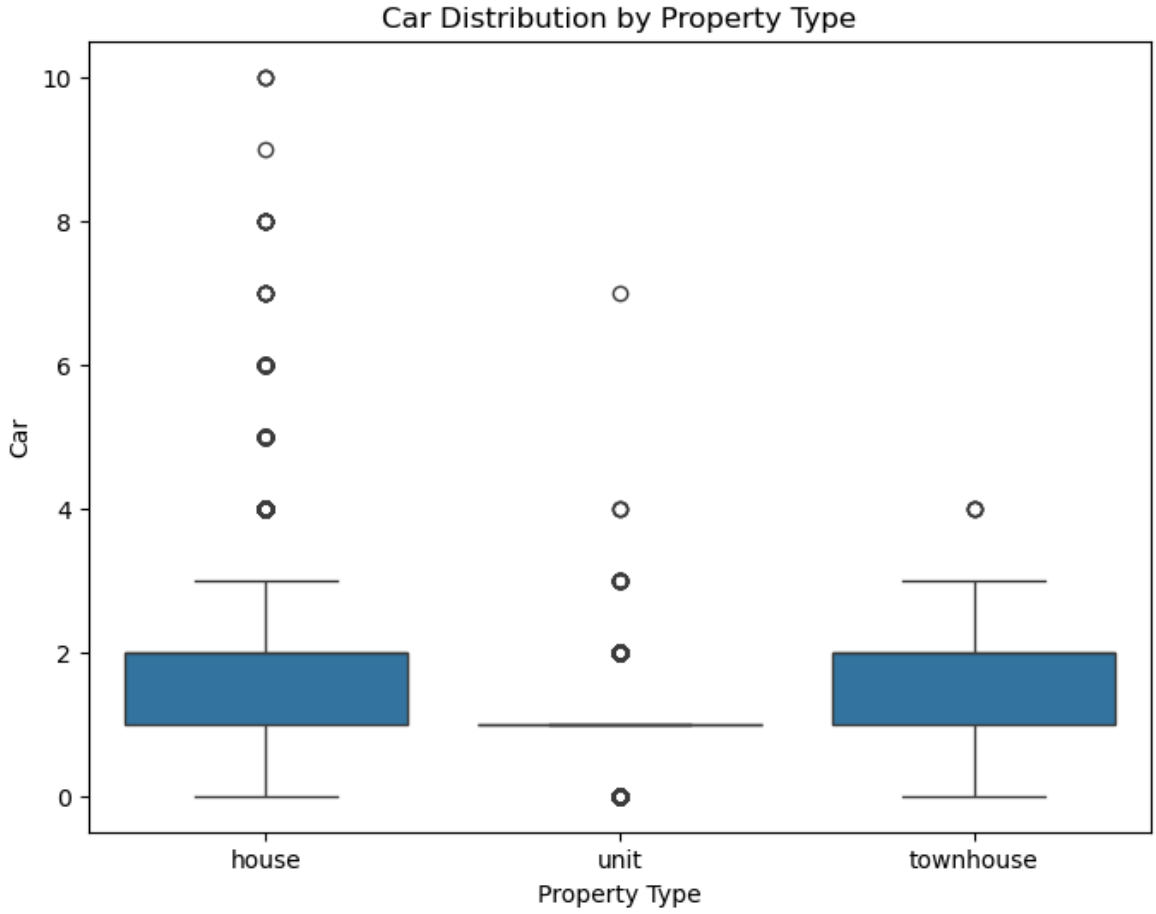
In [118…

```
room_vars = ["Rooms", "Bedroom2", "Bathroom", "Car"]

for var in room_vars:
    plt.figure(figsize=(8, 6))
    sns.boxplot(data=df, x="Type", y=var, order=["house", "unit", "townhouse"])
    plt.title(f"{var} Distribution by Property Type")
    plt.xlabel("Property Type")
    plt.ylabel(var)
    plt.show()
```


Rooms Distribution by Property Type
Rooms
10
8
6
4
2
house
unit
townhouse
Property Type


Bedroom2 Distribution by Property Type
Bedroom2
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
house
unit
townhouse
Property Type





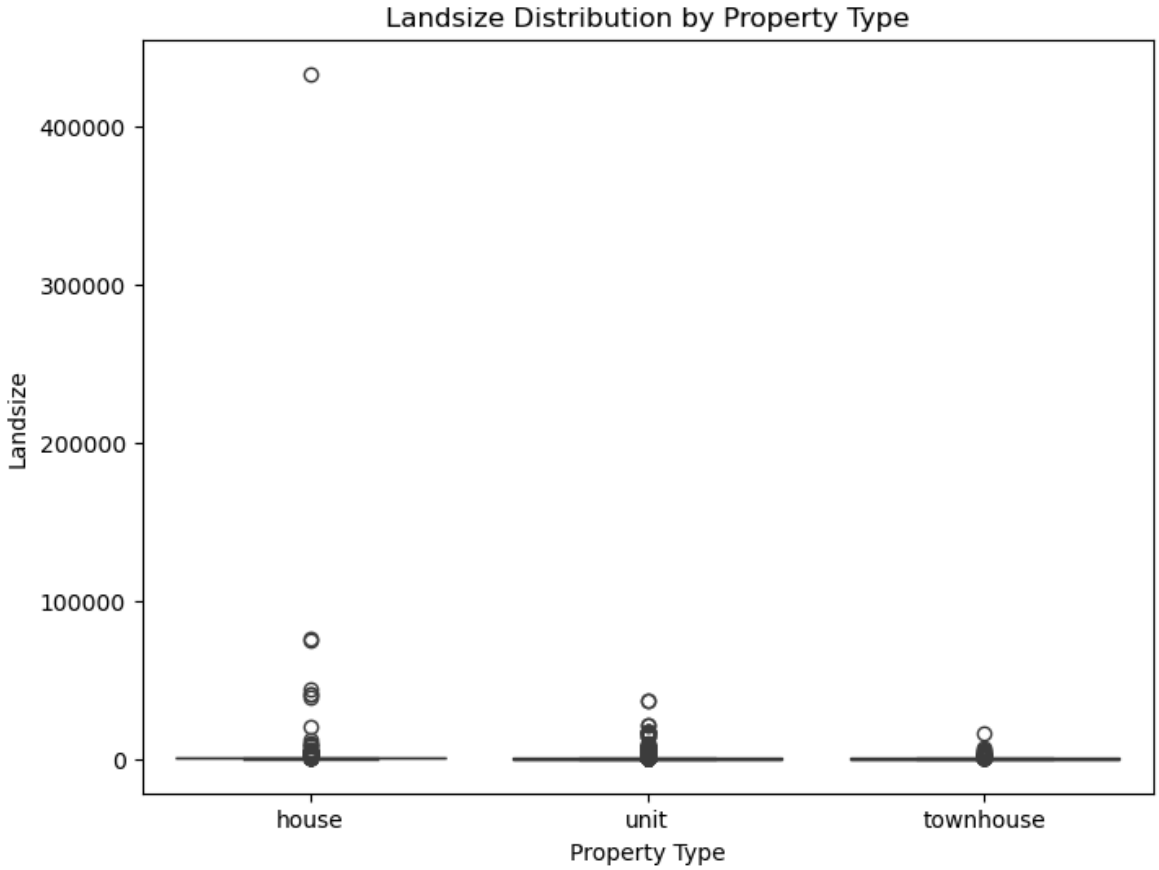
In [119…

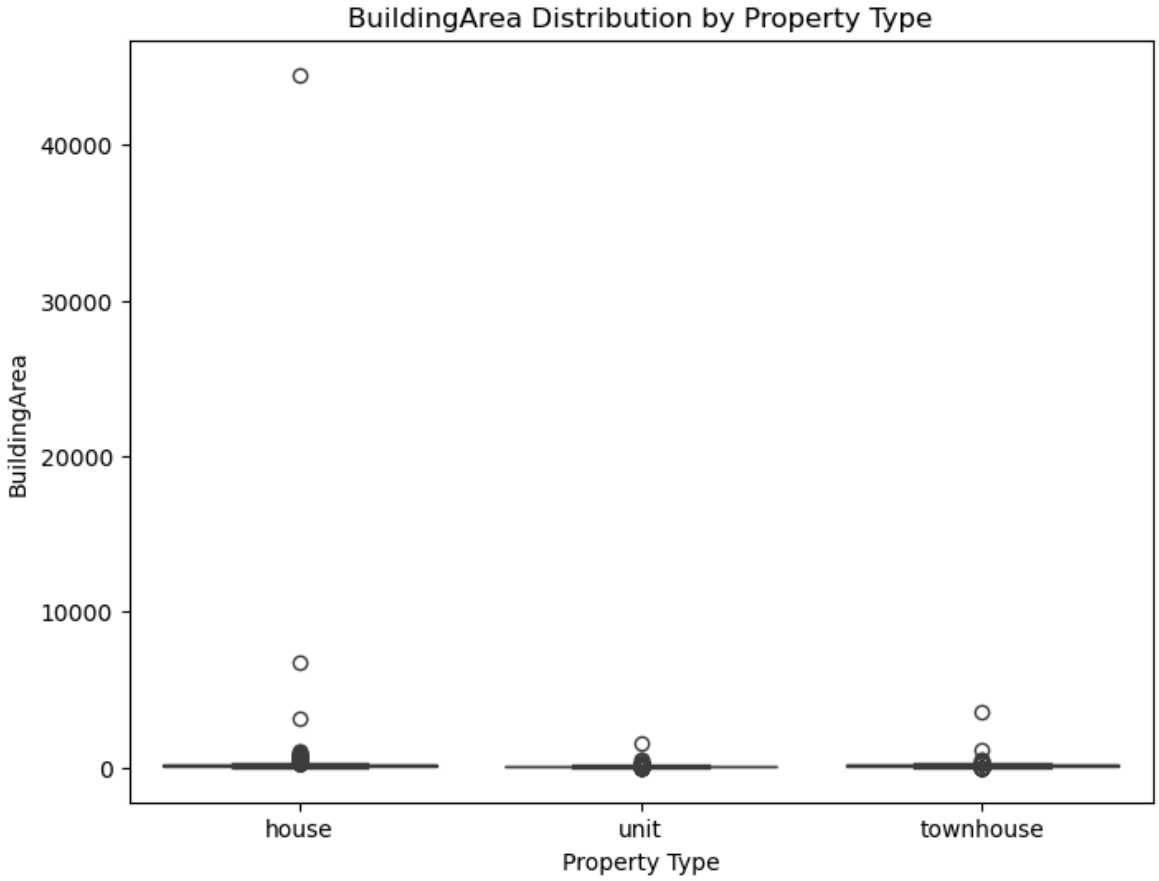
```
room_summary = df.groupby("Type")[["Rooms", "Bedroom2", "Bathroom", "Car"]].medi
print(room_summary)
```

Type	Rooms	Bedroom2	Bathroom	Car
house	3.0	3.0	1.0	2.0
townhouse	3.0	3.0	2.0	2.0
unit	2.0	2.0	1.0	1.0

In [120…

```
size_vars = ["Landsize", "BuildingArea"]

for var in size_vars:
    plt.figure(figsize=(8, 6))
    sns.boxplot(data=df, x="Type", y=var, order=["house", "unit", "townhouse"])
    plt.title(f"{var} Distribution by Property Type")
    plt.xlabel("Property Type")
    plt.ylabel(var)
    plt.show()
```

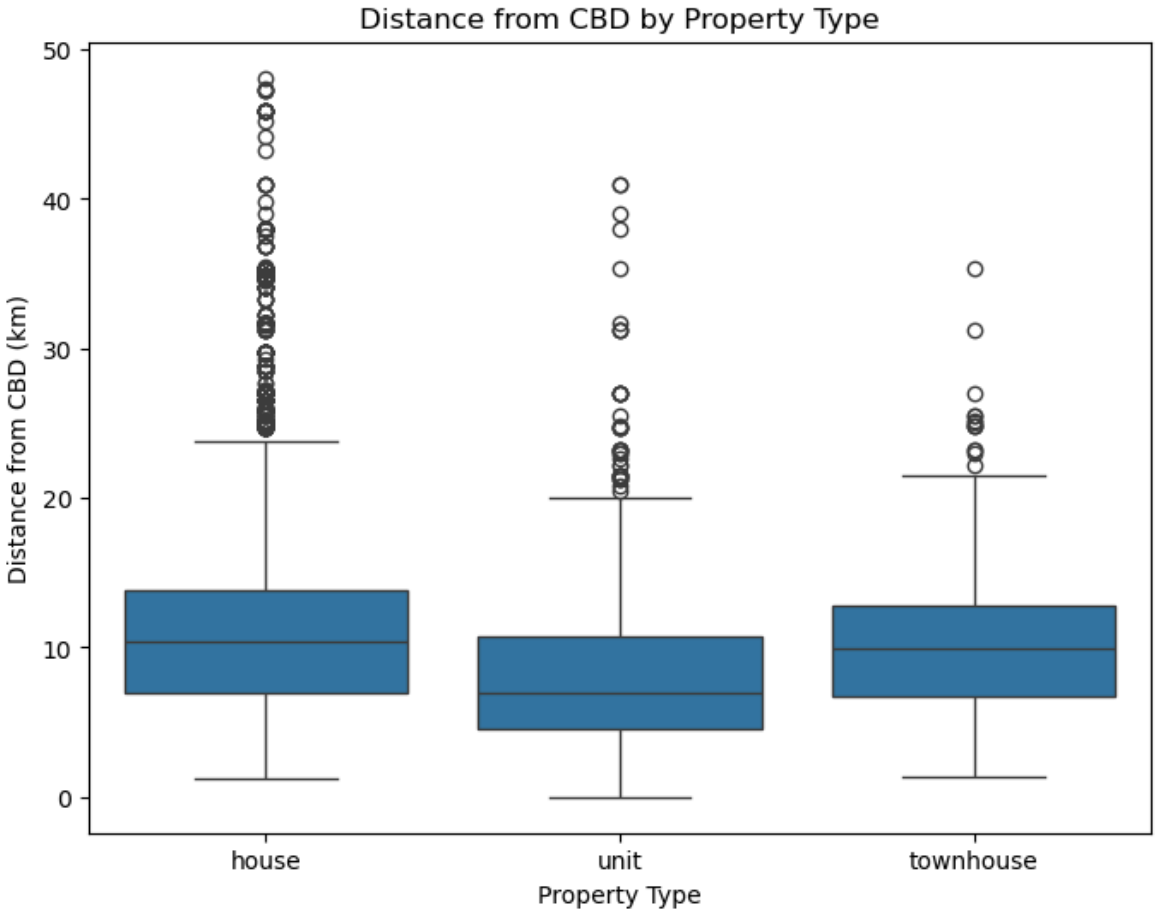




In [121…
```
size_summary = df.groupby("Type")[["Landsize", "BuildingArea"]].median()
print(size_summary)
```

```
           Landsize  BuildingArea
Type
house         560.0         144.0
townhouse     198.5         130.0
unit            0.0          75.0
```

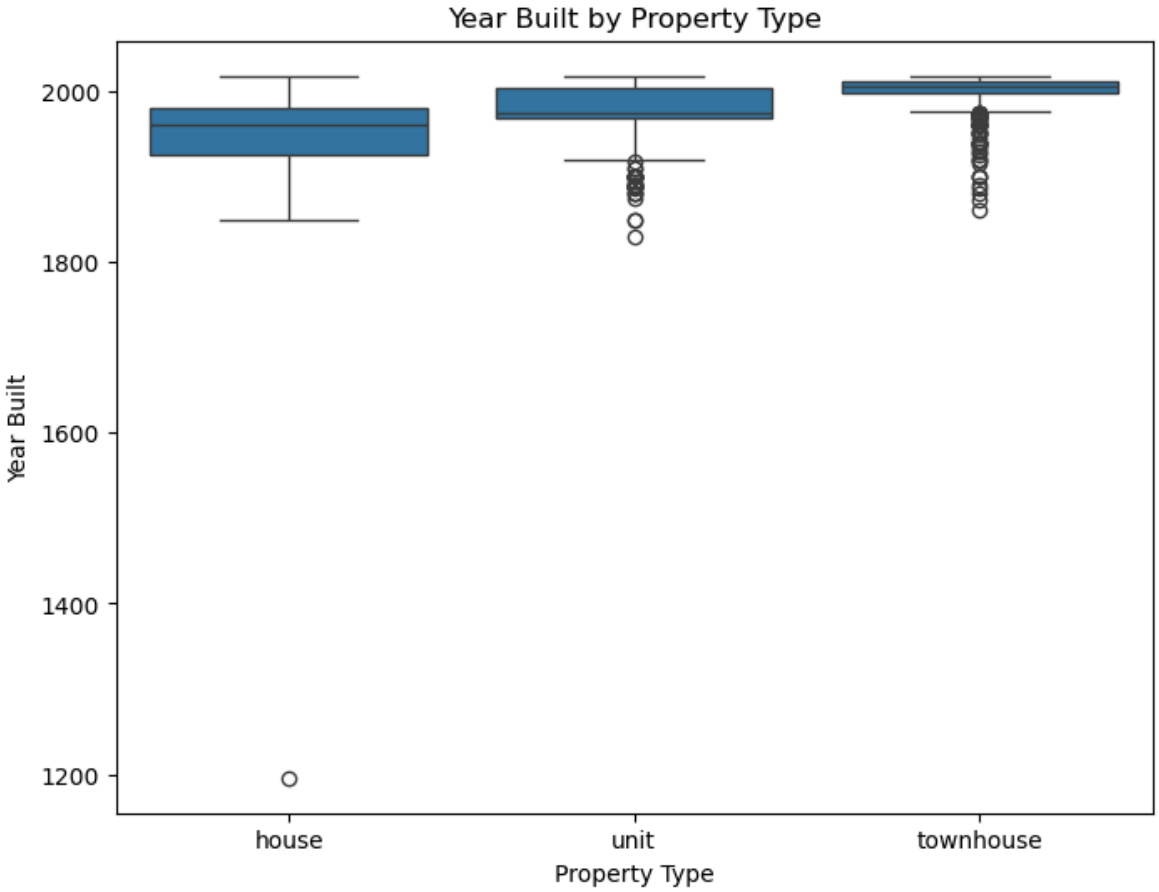
In [122…
```
plt.figure(figsize=(8, 6))
sns.boxplot(data=df, x="Type", y="Distance", order=["house", "unit", "townhouse"
plt.title("Distance from CBD by Property Type")
plt.xlabel("Property Type")
plt.ylabel("Distance from CBD (km)")
plt.show()
```



In [123…
```
distance_by_type = df.groupby("Type")["Distance"].agg(["mean", "median"])
print(distance_by_type)
```

```
                mean  median
Type
house      10.979479    10.4
townhouse   9.851346     9.9
unit        7.607391     6.9
```

In [124…
```
plt.figure(figsize=(8, 6))
sns.boxplot(data=df, x="Type", y="YearBuilt", order=["house", "unit", "townhouse
plt.title("Year Built by Property Type")
plt.xlabel("Property Type")
plt.ylabel("Year Built")
plt.show()
```



In [125…
```
year_summary = df.groupby("Type")["YearBuilt"].median()
print(year_summary)
```

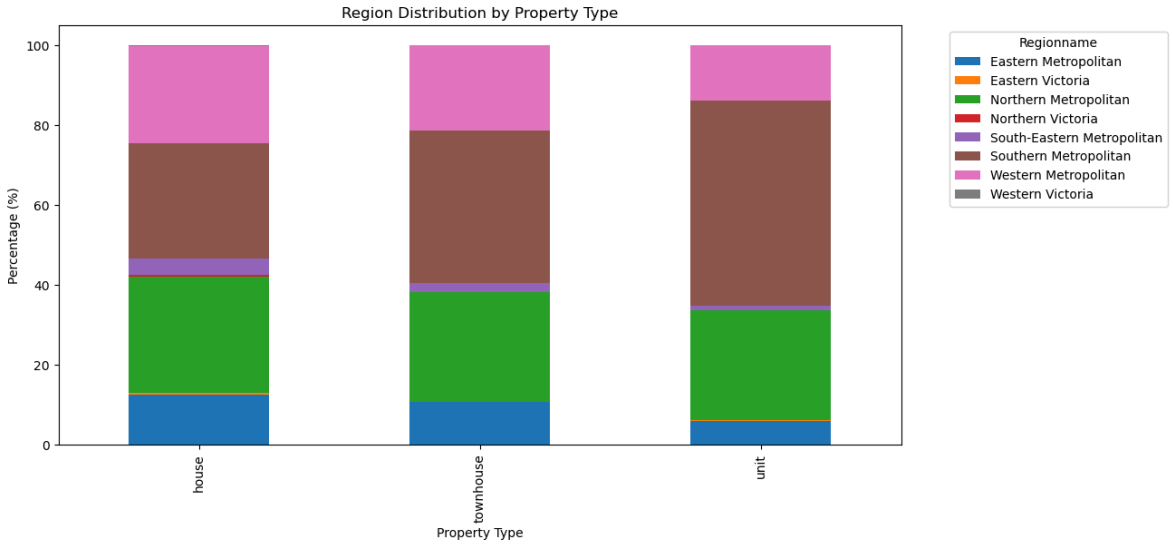
```
Type
house        1960.0
townhouse    2005.0
unit         1975.0
Name: YearBuilt, dtype: float64
```

In [126…
```
region_type = pd.crosstab(
    df["Type"],
    df["Regionname"],
    normalize="index"
) * 100

print(region_type.round(2))
```

```
Regionname  Eastern Metropolitan  Eastern Victoria  Northern Metropolitan  \
Type
house                      12.41              0.53                  29.15
townhouse                  10.59              0.00                  27.56
unit                        5.97              0.10                  27.48

Regionname  Northern Victoria  South-Eastern Metropolitan  \
Type
house                    0.43                        4.11
townhouse                0.00                        2.24
unit                     0.00                        1.23

Regionname  Southern Metropolitan  Western Metropolitan  Western Victoria
Type
house                       28.80                 24.24              0.34
townhouse                   38.15                 21.45              0.00
unit                        51.34                 13.89              0.00
```

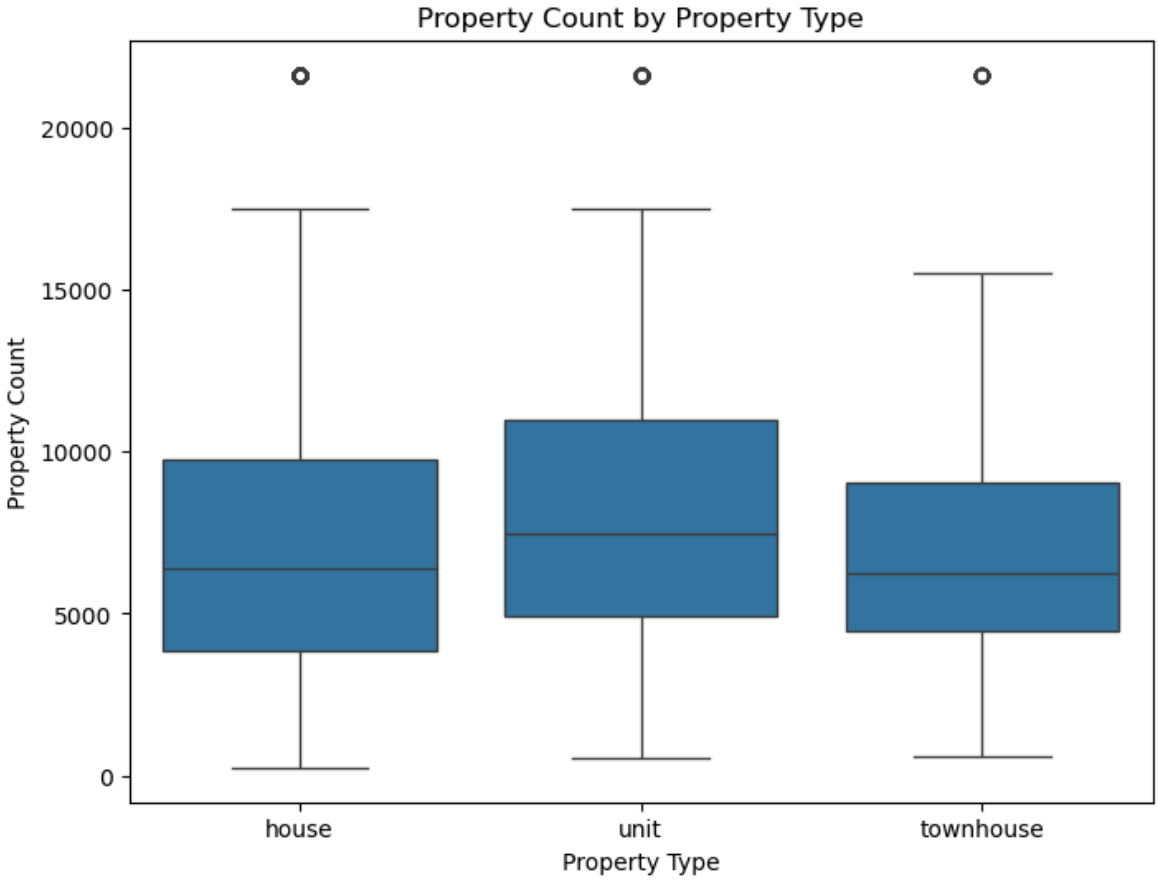
In [127…

```
region_type.plot(kind="bar", stacked=True, figsize=(12, 6))
plt.title("Region Distribution by Property Type")
plt.xlabel("Property Type")
plt.ylabel("Percentage (%)")
plt.legend(title="Regionname", bbox_to_anchor=(1.05, 1), loc="upper left")
plt.show()
```



In [128…

```
plt.figure(figsize=(8, 6))
sns.boxplot(data=df, x="Type", y="Propertycount", order=["house", "unit", "townh
plt.title("Property Count by Property Type")
plt.xlabel("Property Type")
plt.ylabel("Property Count")
plt.show()
```



In [129…
```
propertycount_summary = df.groupby("Type")["Propertycount"].median()
print(propertycount_summary)
```

```
Type
house        6380.0
townhouse    6244.0
unit         7485.0
Name: Propertycount, dtype: float64
```

In [130…
```
type_summary = df.groupby("Type").agg(
    count=("Price", "count"),
    median_price=("Price", "median"),
    mean_price=("Price", "mean"),
    median_rooms=("Rooms", "median"),
    median_bedrooms=("Bedroom2", "median"),
    median_bathrooms=("Bathroom", "median"),
    median_car=("Car", "median"),
    median_landsize=("Landsize", "median"),
    median_building_area=("BuildingArea", "median"),
    median_distance=("Distance", "median"),
    median_year_built=("YearBuilt", "median"),
    median_propertycount=("Propertycount", "median")
)

print(type_summary.round(2))
```

Type	count	median_price	mean_price	median_rooms	median_bedrooms	\
house	9449	1080000.0	1242664.76	3.0	3.0	
townhouse	1114	846750.0	933735.05	3.0	3.0	
unit	3017	560000.0	605127.48	2.0	2.0	

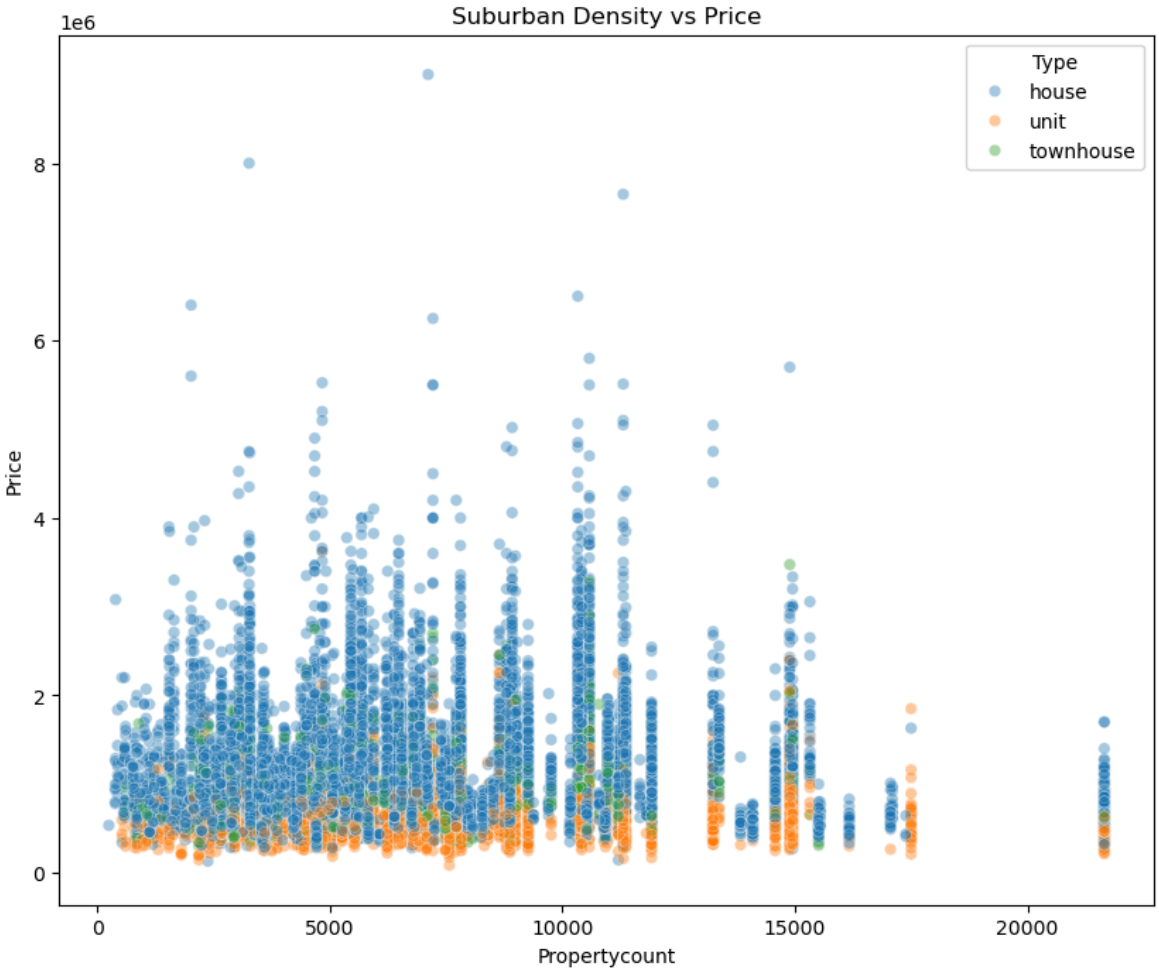
Type	median_bathrooms	median_car	median_landsize	\
house	1.0	2.0	560.0	
townhouse	2.0	2.0	198.5	
unit	1.0	1.0	0.0	

Type	median_building_area	median_distance	median_year_built	\
house	144.0	10.4	1960.0	
townhouse	130.0	9.9	2005.0	
unit	75.0	6.9	1975.0	

Type	median_propertycount
house	6380.0
townhouse	6244.0
unit	7485.0

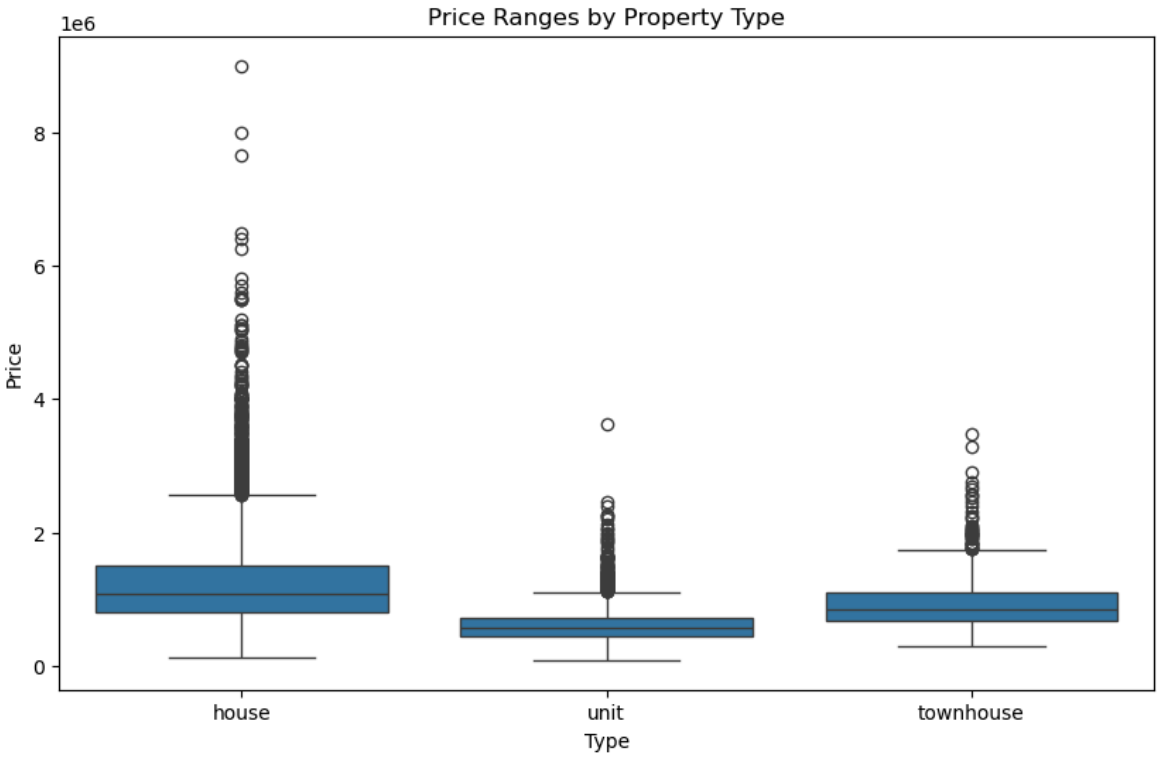
In [131…

```
plt.figure(figsize=(10, 8))
sns.scatterplot(data=df, x="Propertycount", y="Price", hue="Type", alpha=0.4)
plt.title("Suburban Density vs Price")
plt.show()
```



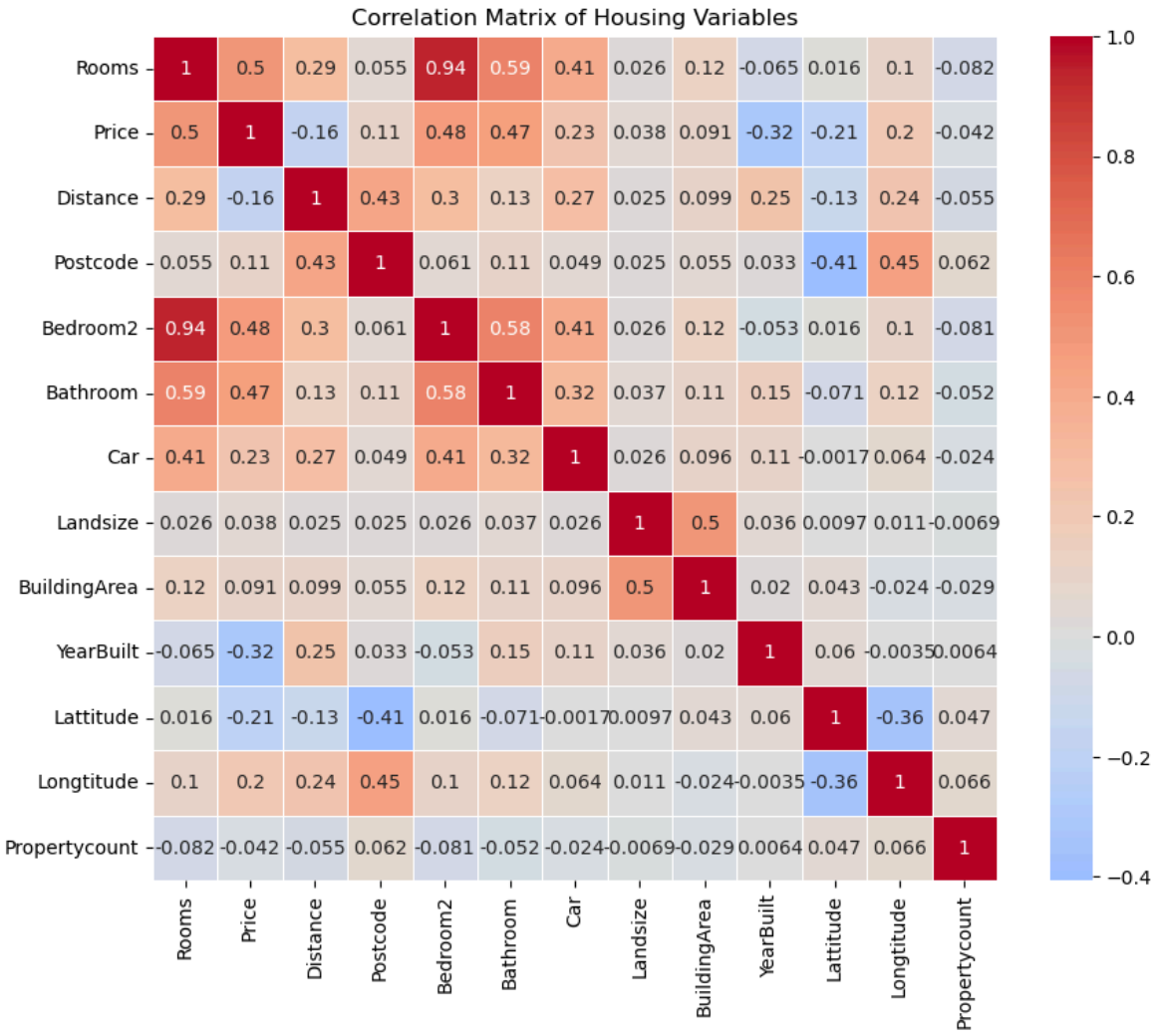
In [132…

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x="Type", y="Price")
plt.title("Price Ranges by Property Type")
plt.show()
```



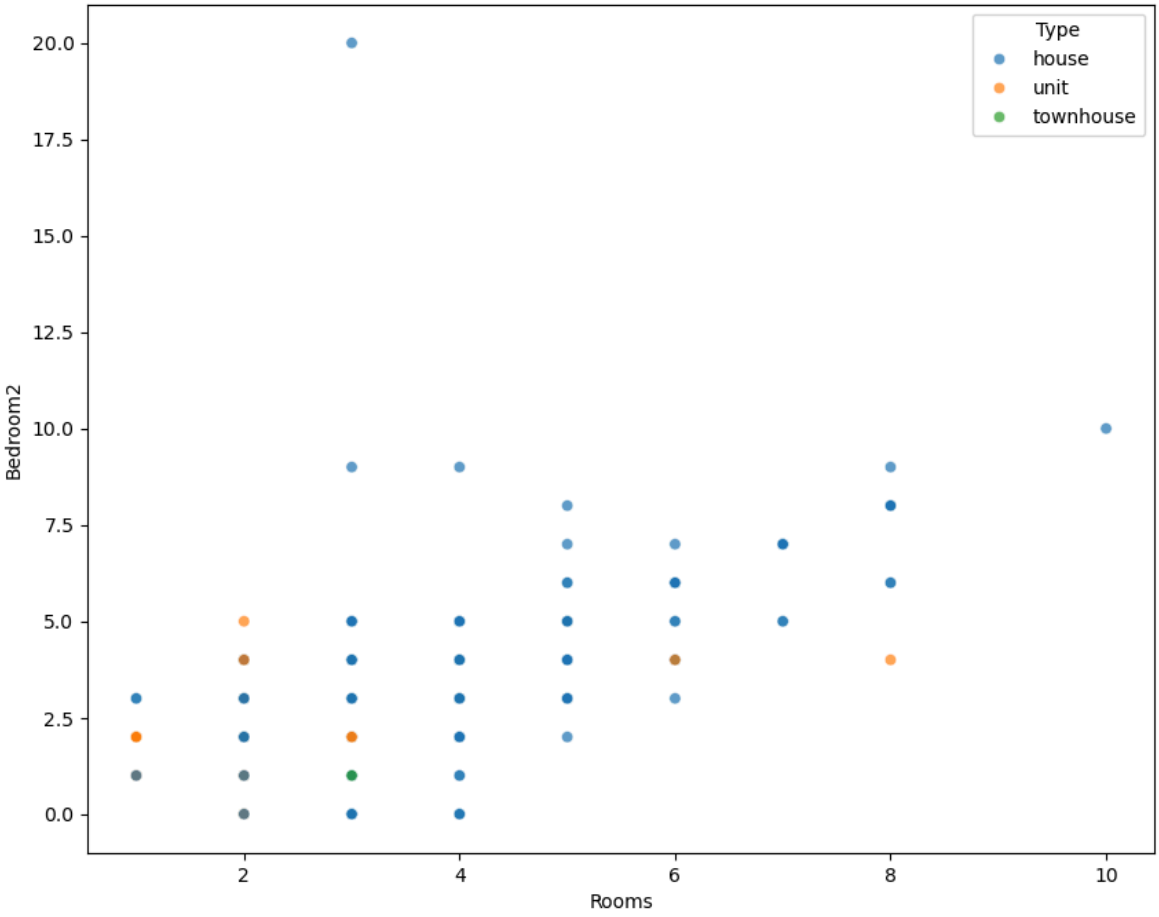
In [133…

```
corr = df.corr(numeric_only=True)

plt.figure(figsize=(10, 8))
sns.heatmap(
    corr,
    annot=True,
    center=0,
    square=True,
    linewidths=0.5,
    cmap="coolwarm"
)
plt.title("Correlation Matrix of Housing Variables")
plt.show()
```



In [134…

```
plt.figure(figsize=(10, 8))
sns.scatterplot(data=df, x="Rooms", y="Bedroom2", hue="Type", alpha=0.7)
plt.show()
```


Type
house
unit
townhouse
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
Bedroom2
2
4
6
8
10
Rooms